

COMP1531 · MAJOR PROJECT

Unigotchi — Iteration 2

Building a Web Server · Ship as a team

```
$ npm start # your Iter 1 functions are now an HTTP API
```

The requirements have changed

Unigotchi blew up beyond UNSW – it now needs a web interface. Same core, new delivery.

ITERATION 1 (done)

Plain JavaScript functions
Called directly, through the interface
Tested with vitest
Data lives in memory (resets each run)
studentId passed in as an argument

ITERATION 2 (now)

Wrap your logic in HTTP routes (Express)
Convert the codebase to TypeScript
Sessions replace passing studentId around
Persist data – survive a server restart
Test over **HTTP**, black-box, via the network

Adapting existing code to changed requirements is a core software-engineering skill – most of your marks come from how you design & demo YOUR routes.

What's new in the toolchain

TypeScript + an Express server + a documented API contract.

TYPESCRIPT

Files become `.ts`
Types on params & returns
`npm run tsc` must pass
`npm run lint` to the style guide
Checked at the Wk 8 check-in

EXPRESS SERVER

`src/server.ts` – routes
`src/config.json` host + port
`npm start` runs the server
Do NOT move config.json
or server.ts

THE API CONTRACT

`swagger.yaml` = your API design
Copy it into your repo
Prescribed routes are given
You ADD your designed routes
HTTP tests via `sync-request-curl`

Reuse Iter 1 logic – the HTTP layer is a thin wrapper. Keep backend functions; routes call them.

Core idea: user sessions

A `studentId` identifies a student – it is NOT proof the caller IS that student.

THE PROBLEM

Student IDs are visible & guessable
GET `/students/z1234567/...` could
leak another user's data

A session token is the proof.

Random, hard-to-guess, returned on
register / login.

WHAT A ROUTE DOES

1. Read the `session` header
2. Missing/invalid → **401**
3. Resolve session → `studentId`
4. Check they may access the resource
5. Call your Iter 1 logic with that id

Session goes in a `header`, never the URL

Keep `studentId` params in backend functions – the route layer gets the id from the session instead of trusting the client.

Errors, persistence & CI

HTTP status codes, data that survives a restart, and a pipeline that catches mistakes.

ERROR RESPONSES

401 no / invalid session
403 valid session, not allowed
400 bad user input

Order: **401** → **403** → **400**
then first-listed in swagger

EXCEPTIONS

```
throw new UnigotchiError(  
    'INVALID_STUDENT_ID', ...)
```

Catch at the route layer,
map to a status + JSON:
{ error, message }

OPTIONAL

PERSISTENCE + CI

State survives a **restart**
No external database
DELETE /clear resets +
writes cleared state

Pipeline: **lint** + **tsc** + tests

Passwords are stored hashed (never plaintext); sessions are random tokens. clear must not delete starter or universe files.

The prescribed routes (automarked)

A deliberately small shared core, in `swagger.yaml`. Everything else you design.

Method + Route	Sends	200 returns	Errors
POST /auth/register	studentId, password, nameFirst, nameLast	{ session }	400 INVALID_STUDENT_ID / _FIRST_NAME / _LAST_NAME / _PASSWORD
POST /auth/login	studentId, password	{ session }	400 INVALID_CREDENTIALS
POST /simulations	session hdr + universeData	{ simulationId }	401 UNAUTHORISED · 400 INVALID_UNIVERSE / MAX_SIMULATIONS
GET /simulations/{id}	session hdr	status, currentActivity, energy, happiness, timeElapsed	401 UNAUTHORISED · 403 INVALID_SIMULATION
DELETE /clear	– (no session)	{ }	–

Session header carries the logged-in user. /clear and /auth/* need no session; simulation routes do.

Your routes: the Student Wellbeing Advisor

Design, document, test & demo these 6 stories.

1 • PLAN A WEEK

Compare ≥ 3 next actions with cost / benefit / risk; flag a current risk or blocker

2 • SOCIAL SUPPORT

Create / inspect / remove connections; no duplicates; affects later behaviour

3 • WELLBEING ACTIVITIES

≥ 2 new non-study activities; one needs social, one needs an opportunity

4 • OPPORTUNITIES

Clubs / societies from the universe; join & leave; max 3; benefits apply later

5 • EXAM READINESS

HIGH / MED / LOW per exam, deterministic, documented thresholds

6 • RECOMMEND NEXT

Never suggests an invalid action; gives a reason + an alternative

Add designed routes to swagger AFTER the prescribed ones • build a traceability table in features.md • must be state-driven, not hardcoded.

Live demo

First: see where this is going. Then: build a route the same way – red → green.

SEE THE TARGET (26T1 frontend)

Your routes are what power this.

BUILD IT (HTTP, test-first)

Demo test: **POST** /auth/register
expect **200** + { session } → RED

Implement the route in server.ts → **GREEN**

Also poke it live with **curl** / Postman
Test the **status**, the body, & side effects
DELETE /clear in beforeEach

APIs are cool!

How a team ships a route

1

design

2

branch

3

write HTTP tests

4

implement route

5

open MR

6

peer review

7

creator merges

THE RULES

`master` is protected

Reviewed by someone who didn't write it

Never merge failing / untested / unapproved code

= **penalised**

Write some tests before implementing

WORKING IN PARALLEL

Design swagger first – unblocks everyone

Downstream work uses stubbed routes

`git pull origin <branch>` to share WIP

Pull master regularly

Each member: MRs on prescribed + designed

Regular standups (≥ 2 / week) on MS Teams • GitLab issues with acceptance criteria written before you code.

Where the marks are

Design Check-in & Final Demo

50%

Automarking (prescribed routes)

20%

Group Tests + coverage

10%

Code Quality

10%

Git, PM & Teamwork

10%

THE AUTOMARK FORMULA

$$\text{Mark} = t \cdot i \cdot \min((c+1)/100, 1)^3$$

t = your tests vs your code

i = hidden course tests vs your code

c = statement coverage (+1, cap 100)

→ 99% coverage = full coverage marks

You CANNOT pass Iter 2 on automarked prescribed routes alone – the design check-in & demo (50%) are required.
Bonus: up to 5%.

The timeline

Two deadlines. Get TypeScript + prescribed routes done early, then design & demo.

WEEK 8 CHECK-IN

Due **12pm Monday, Week 8** · your Wk 8 lab

Iter 1 logic → HTTP routes in server.ts
TypeScript conversion done (**tsc** passes)
swagger.yaml API design presented
GitLab pipeline set up
Evidence coverage has been run

WEEK 10 FINAL

Due **9am Monday, Week 10** · Wk 10 lab

All designed routes implemented + tested
Demo prescribed + designed routes with
Postman / Insomnia / curl
Traceability table in features.md
Explain readiness & recommendation models

No late submissions on the check-in. Final: up to 48h late (core only) at 5% / 24h. Pushing to master = submitting.

Before you submit

DRYRUN & PREVIEWS

```
cd project-backend
```

```
1531 dryrun 2
```

Checks return shapes + basic behaviour

– not correctness or your tests

Previews run Wed, Fri, Sun before submission

DEADLINES

12pm Mon Wk 8 check-in (10%)

9am Mon Wk 10 final (50%)

Demoed in YOUR lab · all members attend

THE PATH

Convert to TypeScript → wrap logic in Express routes → sessions + persistence → design your advisor routes in swagger → test over HTTP → demo with the frontend & Postman. Read spec **section 4 & 5** in full this week.

Keep master green – it IS your submission.